

Национальный исследовательский университет ИТМО
Факультет программной инженерии и компьютерной техники

Отчёт по лабораторной работе №2
по дисциплине «Архитектура программных систем»

Выполнил:

Студент группы Р3316

Брагин Роман Андреевич

Преподаватель:

Перл Иван Андреевич

Санкт-Петербург

2025 г.

Задание

Из списка шаблонов проектирования GoF и GRASP выбрать 3–4 шаблона и для каждого из них придумать 2–3 сценария, для решения которых могут быть применены выбранные шаблоны.

Сделать предположение о возможных ограничениях, к которым может привести использование шаблона в каждом описанном случае. Обязательно выбрать шаблоны из обоих списков.

Выбор шаблонов проектирования

Для выполнения лабораторной работы были выбраны шаблоны проектирования из списков GoF и GRASP, обеспечивающие рассмотрение как объектно-ориентированных, так и архитектурных принципов проектирования:

- из группы GoF: *Factory Method* (Фабричный метод), *Decorator* (Декоратор);
- из группы GRASP: *Low Coupling* (Низкая связность), *High Cohesion* (Высокая связность).

Выбранные шаблоны широко применяются при проектировании масштабируемых и расширяемых программных систем и позволяют продемонстрировать различные аспекты ответственности, расширяемости и поддержки кода.

1 Шаблон Factory Method (Фабричный метод) — GoF

Описание шаблона

Шаблон Factory Method определяет интерфейс для создания объектов, позволяя подклассам изменять тип создаваемых экземпляров. Основная цель

шаблона — инкапсуляция процесса создания объектов и снижение зависимости клиентского кода от конкретных классов.

Сценарии применения

Сценарий 1. Создание объектов уведомлений

В корпоративной системе оповещений поддерживается несколько каналов доставки сообщений: электронная почта, SMS, push-уведомления и сообщения в мессенджерах. Каждый тип уведомления требует собственной реализации, зависящей от внешних API и форматов сообщений.

Использование шаблона Factory Method позволяет вынести логику создания конкретных уведомлений в специализированные фабрики. Клиентский код работает с абстрактным интерфейсом уведомления, не зная деталей его реализации, что упрощает добавление новых каналов доставки.

Сценарий 2. Инициализация элементов пользовательского интерфейса

В кроссплатформенном приложении требуется создавать элементы интерфейса, адаптированные под различные операционные системы (Windows, Linux, macOS). Конкретная реализация кнопок, окон и диалогов отличается в зависимости от платформы.

Фабричный метод позволяет делегировать создание UI-компонентов специализированным фабрикам, сохраняя единый интерфейс для их использования и изолируя платформозависимый код.

Сценарий 3. Создание обработчиков файлов различных форматов

Система обработки документов поддерживает работу с файлами форматов PDF, DOCX и TXT. Каждый формат требует собственного алгоритма чтения, валидации и парсинга.

Применение Factory Method позволяет создавать соответствующий об-

работчик файла на основании его типа, не модифицируя код клиента при добавлении новых форматов.

Возможные ограничения

- увеличение количества классов за счёт введения отдельных фабрик;
- усложнение структуры проекта при большом числе типов создаваемых объектов;
- необходимость проектирования корректной иерархии абстракций.

2 Шаблон Decorator (Декоратор) — GoF

Описание шаблона

Шаблон Decorator позволяет динамически добавлять объектам новую функциональность, оборачивая их в дополнительные классы-декораторы. Шаблон предоставляет гибкую альтернативу наследованию для расширения поведения объектов.

Сценарии применения

Сценарий 1. Расширение функциональности логирования

В серверном приложении используется базовый механизм логирования. В зависимости от конфигурации системы требуется добавлять дополнительные возможности: запись логов в файл, отправку логов во внешние системы мониторинга, шифрование логируемых данных.

Использование декораторов позволяет поэтапно расширять поведение логгера без изменения его базовой реализации и без создания громоздкой иерархии наследования.

Сценарий 2. Формирование HTTP-ответов

При обработке HTTP-запросов может потребоваться добавление различных аспектов к ответу: сжатие данных, кэширование, добавление заголовков безопасности, логирование времени обработки.

Каждый аспект реализуется в виде отдельного декоратора, что позволяет гибко комбинировать поведение в зависимости от требований конкретного запроса.

Сценарий 3. Модификация бизнес-операций

В финансовых системах операции могут дополняться проверками прав доступа, аудитом, расчетом комиссий и логированием. Применение шаблона Decorator позволяет добавлять эти функции без изменения основной бизнес-логики операции.

Возможные ограничения

- усложнение отладки из-за большого числа обёрток;
- увеличение количества объектов во время выполнения программы;
- снижение читаемости кода при чрезмерном использовании декораторов.

3 Шаблон Low Coupling (Низкая связность) — GRASP

Описание шаблона

Принцип Low Coupling рекомендует проектировать классы с минимальным количеством зависимостей друг от друга. Низкая связность снижает влияние изменений в одной части системы на другие компоненты и упрощает сопровождение кода.

Сценарии применения

Сценарий 1. Интеграция с внешними сервисами

При взаимодействии с платёжными или аналитическими сервисами используется слой абстракции в виде интерфейсов и адаптеров. Основная бизнес-логика системы не зависит от конкретных реализаций внешних API.

Это позволяет заменять поставщиков сервисов или изменять протоколы взаимодействия без значительных изменений в кодовой базе.

Сценарий 2. Архитектура микросервисов

Каждый микросервис выполняет ограниченный набор функций и взаимодействует с другими сервисами через чётко определённые контракты. Принцип низкой связности снижает риск каскадных отказов и упрощает масштабирование системы.

Сценарий 3. Модульное тестирование

Слабая связность компонентов позволяет изолировать отдельные модули при тестировании, используя заглушки и моки. Это повышает надёжность тестов и ускоряет процесс разработки.

Возможные ограничения

- увеличение количества интерфейсов и вспомогательных классов;
- дополнительные накладные расходы на проектирование архитектуры;
- риск избыточной абстракции при неправильном применении принципа.

4 Шаблон High Cohesion (Высокая связность) — GRASP

Описание шаблона

Принцип High Cohesion предполагает, что каждый класс или модуль должен иметь чётко определённую и узкую зону ответственности. Высокая связность повышает читаемость, поддерживаемость и переиспользуемость компонентов.

Сценарии применения

Сценарий 1. Разделение бизнес-логики и инфраструктуры

В многоуровневой архитектуре бизнес-логика, работа с базой данных и обработка HTTP-запросов размещаются в отдельных модулях. Каждый модуль выполняет строго определённую функцию и не содержит несвязанных обязанностей.

Сценарий 2. Проектирование доменных объектов

Доменные сущности инкапсулируют поведение, напрямую связанное с их данными. Например, объект заказа отвечает за расчёт стоимости и изменение статуса, но не за отображение или хранение в базе данных.

Сценарий 3. Организация сервисного слоя

Каждый сервис реализует конкретный бизнес-сценарий и не объединяет несвязанные операции. Это упрощает тестирование, сопровождение и повторное использование сервисов.

Возможные ограничения

- увеличение количества классов при строгом разделении обязанностей;
- необходимость тщательного анализа ответственности на этапе проек-

тирования;

- возможное усложнение навигации по проекту.

Вывод

В ходе лабораторной работы были рассмотрены шаблоны проектирования из списков GoF и GRASP, охватывающие как объектно-ориентированные паттерны, так и архитектурные принципы проектирования. Для каждого шаблона были приведены детализированные технические сценарии применения и проанализированы возможные ограничения. Осознанное использование шаблонов проектирования позволяет создавать гибкие, расширяемые и сопровождаемые программные системы.